

Projet : Générateur de Datasets

Objectif du projet

L'objectif de ce projet est de concevoir un **système logiciel modulaire et extensible** permettant de **générer des datasets sur mesure** à partir d'une **définition formelle d'un projet de données**. Le système devra être pensé pour respecter les **principes SOLID** et présenter une **modélisation UML claire**, notamment un **diagramme de classes complet** et cohérent.

Ce projet a pour vocation de vous exercer à :

- Appliquer les **principes de conception orientée objet (SOLID)**.
- Concevoir une **architecture évolutive et maintenable**
- **Appliquer ces concepts** sur un projet (Spring)
- Mettre en place une **API** qui **respecte la définition de REST**
- Votre production devra permettre de :
 - Définir des **projets de datasets** (ex : population, e-commerce...)
 - Définir des **entités** et leurs **attributs**
 - Générer automatiquement des données fictives
 - Exporter les résultats dans plusieurs formats (CSV, JSON...)

Contexte

Dans le domaine du **Big Data** et de **l'analyse de données**, il est souvent difficile de trouver des datasets adaptés pour les tests, les démonstrations ou les projets d'apprentissage automatique. L'idée est de proposer une **solution logicielle capable de générer automatiquement des datasets réalistes** à partir de définitions paramétrables.

Un utilisateur pourra :

1. **Créer un projet de dataset** (ex : “Population française”, “Ventes e-commerce”, “Capteurs IoT”).
2. **Définir les entités** du dataset (ex : Utilisateur, Produit, Capteur).
3. Pour chaque entité, **définir les attributs** :
 - nom
 - type de donnée (String, Integer, Float, Boolean, Date, Enum, etc.)
 - contraintes (bornes min/max, moyenne, médiane, valeurs possibles, distribution des valeurs).
4. **Créer des sous-entités** (ex : une entité Adresse rattachée à Utilisateur).
5. **Configurer la taille du dataset**.
6. **Exporter le dataset** dans différents formats (CSV, JSON, XML, SQL...).

Le système devra permettre d'étendre facilement les fonctionnalités (principe **Open/Closed**) : nouveaux types de données, nouveaux formats d'export, nouvelles sources de données.

Fonctionnalités principales attendues

1. Gestion des projets de dataset

- Création, édition et suppression d'un projet.
- Chaque projet contient un ensemble d'entités.

2. Gestion des entités et attributs

- Création d'entités avec leurs attributs.
- Gestion des sous-entités (composition ou agrégation).

- Définition des types de données et contraintes (min, max, moyenne, médiane, distribution).

3. Génération de données

Votre projet devra être capable de générer des données sur modèles :

- Génération aléatoire respectant les contraintes définies.
- Génération à partir de **sources externes** :
 - API de prénoms/noms réalistes (ex : français, internationaux).
 - Listes de villes, professions, produits, etc.

4. Formats d'export

- CSV
- JSON
- XML
- SQL

Les formats doivent pouvoir être ajoutés facilement sans modification du cœur de l'application.

Contraintes techniques et pédagogiques

- A partir de votre production, vous devrez produire un diagramme **cohérent, complet et lisible** qui présente l'ensemble du contexte.
- L'architecture doit pouvoir **évoluer sans modification majeure du code existant** (principe Open/Closed).
- Les classes doivent être **faiblement couplées et fortement cohésives**.
- L'héritage doit être utilisé avec discernement (Liskov, Interface Segregation).

- La dépendance envers les implémentations concrètes doit être limitée (Dependency Inversion).

Le projet

Étape 0 – Initialisation du projet

Objectif : Créer la structure de base du projet et vérifier que tout fonctionne.

Tâches à réaliser

- Créer un projet **Spring Boot** (via <https://start.spring.io>) avec :
 - Spring Web
 - Spring Data JPA
 - Lombok
 - H2 Database
 - (Optionnel) MapStruct
- Configurer `application.yml` avec la base H2
- Créer la structure suivante :

```
src/main/java/com/example/datagenerator
├── controller
├── service
├── repository
├── model
├── dto
└── mapper
```

- Ajouter un `README.md` expliquant :
 - le but du projet
 - comment lancer le projet

Étape validée quand :

- Le projet démarre sans erreur.
- La console H2 est accessible.

Étape 1 — Modélisation de base (Projet / Entités / Attributs)

Objectif : Mettre en place les classes principales pour représenter les projets de datasets.

À faire

- Créer les entités :
 - `Project` (id, name, description)
 - `EntityDefinition` (id, name, lié à un projet)
 - `AttributeDefinition` (id, name, type)
- Le type d'attribut peut être un simple **Integer** pour commencer.
- Créer les **repositories** JPA correspondants
- Créer les **services** pour gérer le CRUD
- Créer les **DTO** et **Mappers** (manuels ou avec MapStruct)
- Créer les **Contrôleurs REST** et implémenter intégralement une API complète sur les endpoints suivants :
 - `/api/projects`

- /api/entities
- /api/attributes
- Créer un **front minimal** :
 - Liste des projets
 - Formulaire pour créer un projet et y ajouter des entités

Étape validée quand :

- Vous pouvez créer un projet, lui ajouter des entités et attributs, et les afficher.

Étape 2 — Gestion des sous-entités

Objectif : Permettre qu'une entité contienne d'autres entités (composition).

À faire

- Ajouter une relation parent/enfant :

```
@ManyToOne
private EntityDefinition parentEntity;

@OneToMany(mappedBy = "parentEntity")
private List<EntityDefinition> subEntities;
```

- Mettre à jour les DTO et mappers
- Ajouter des endpoints REST pour gérer les sous-entités :
 - /api/entities/{id}/subentities
- Adapter le front pour afficher et gérer les sous-entités

Étape validée quand :

- Vous pouvez créer une entité avec des sous-entités (ex : Utilisateur → Adresse).

Étape 3 – Système d'export

Objectif : Générer et exporter les données définies dans les projets.

À faire

- Créer un service `DatasetGeneratorService` avec une méthode :

```
Dataset generate(Project project, int count);
```

- Génération simple : remplir les valeurs avec des données aléatoires
- Créer une interface `Exporter` :

```
public interface Exporter {  
    ... byte[] export(Dataset dataset);  
}
```

- Implémenter `JsonExporter` , nous verrons plus tard pour quelque chose de générique
- Ajouter un endpoint :

```
GET /api/export?projectId=1&format=json
```

- Ajouter un bouton "Exporter" dans le front

Étape validée quand :

- Vous pouvez générer et télécharger un JSON simple.

Étape 4 – Généralisation des attributs

Objectif : Rendre le système extensible et plus réaliste.

À faire

- Créer une énumération `AttributeType` { `STRING`, `INTEGER`, `FLOAT`, `BOOLEAN`, `DATE` } afin de rendre possible l'ajout de n'importe quel type d'attribut.
- Adapter le `DatasetGeneratorService` pour utiliser ces générateurs
- Ajouter un champ `constraints` (min, max...) si vous souhaitez aller plus loin

Étape validée quand :

- Les valeurs générées sont cohérentes selon le type d'attribut.

Étape 5 — Finalisation

Objectif : Nettoyer, documenter et rendre le projet présentable.

À faire

- Documenter les endpoints avec **Swagger / OpenAPI**
- Ajouter la pagination sur les listes si nécessaire
- Corriger les messages d'erreur et exceptions
- Mettre à jour le `README.md` :
 - Instructions de build et exécution
 - Exemple de jeu de données
 - Captures d'écran du front

Étape validée quand :

- Le projet est complet, documenté et facilement testable.

Étapes bonus (optionnelles)

- Ajouter un modèle prédéfini (population française, ventes e-commerce...)
- Ajouter un connecteur externe (API de prénoms, villes...)
- Dockeriser le projet

Conseils finaux

- Progressez **étape par étape**, cochez vos tâches au fur et à mesure ☒
- Testez souvent avec **Postman**, **cURL** ou **Swagger** avant de connecter le front
- Commencez simple (Thymeleaf ou REST pur), puis améliorez
- Gardez votre code **clair et commenté** (quand nécessaire pour vous :))

Livraison

La livraison de votre projet se fera exclusivement sur Github Classroom à cette adresse :

<https://classroom.github.com/a/4nN8oo78>

La livraison de votre projet comptera pour l'évaluation des modules suivants :

- SOLID
- REST

Plus tard, vous devrez incrémenter votre projet avec de nouveaux ajouts sur les thématiques qui seront abordées lors des prochains cours, ce projet servira de base et nous l'amélioreront

Bon code et amusez-vous !